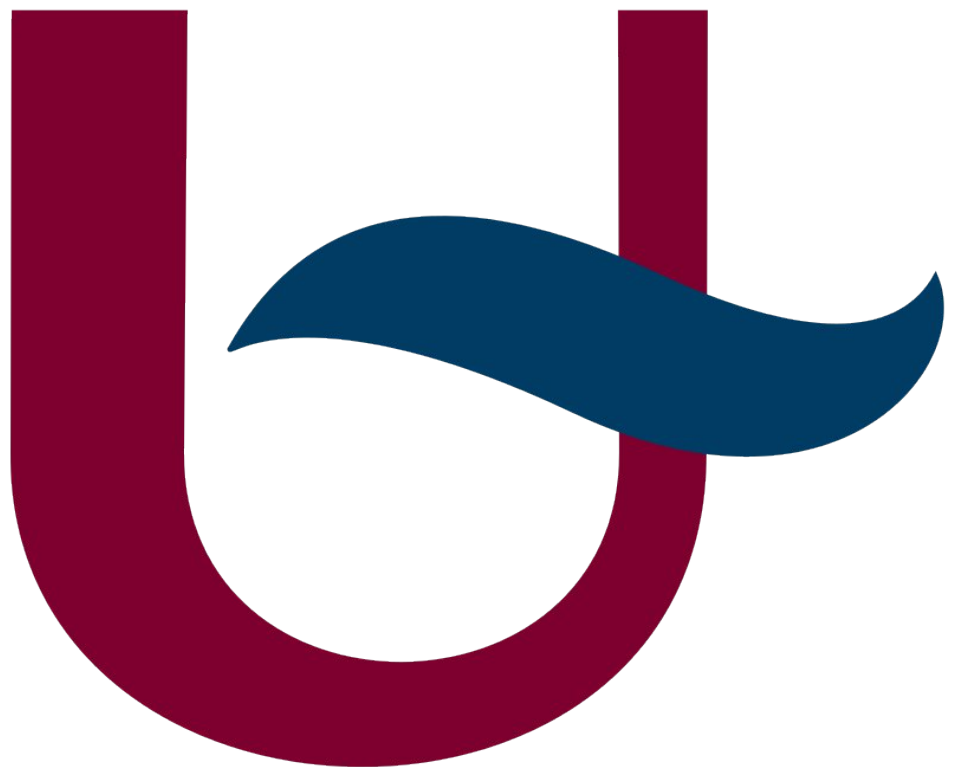


I-MAP: LOGBOOK (AI USAGE)

TSMK



LLM request

△ [Claude](#) was used for this prompt.

I got this struct for a BNE085 IMU sensor: `/** @brief datastruct of IMU measurement data */ struct movement { float roll; /** @brief roll defines rolling*/ float pitch; /** @brief pitch defines nose up or down*/ float yaw; /** @brief yaw defines left right swing*/ float w; /** @brief real part of quaternionv*/ float i; /** @brief i part*/ float j; /** @brief j part*/ float k; /** @brief k part*/ }; typedef struct movement movement_t; But I'm using a PIM448, that provides these values: float getAccelX() { return _accel.x } float getAccelY() { return _accel.y } float getAccelZ() { return _accel.z } float getGyroX() { return _gyro.x } float getGyroY() { return _gyro.y } float getGyroZ() { return _gyro.z } float getMagX() { return _mag.x } float getMagY() { return _mag.y } float getMagZ() { return _mag.z } float getTemperature() { return _temp } float getResultantG() { return _resultantG } How can i convert them?`

LLM resopnse

The PIM448 gives you **raw sensor data** (accelerometer, gyroscope, magnetometer), while the `movement_t` struct expects **fused orientation data** (Euler angles + quaternion). You need a sensor fusion algorithm to bridge them.

The Core Problem

PIM448 output	<code>movement_t</code> needs
Raw accel XYZ	roll, pitch, yaw
Raw gyro XYZ	w, i, j, k (quaternion)
Raw mag XYZ	(used for yaw/heading)

Option 1: Mahony Filter (Recommended — lightweight, runs on MCUs)

```
c
#include <math.h>

// Mahony filter state
static float q0 = 1.0f, q1 = 0.0f, q2 = 0.0f, q3 = 0.0f;
static float integralFBx = 0.0f, integralFBy = 0.0f, integralFBz = 0.0f;

#define SAMPLE_FREQ 100.0f // Hz — match your actual update rate
#define TWO_KP 2.0f // Proportional gain
#define TWO_KI 0.005f // Integral gain

static float invSqrt(float x) {
    return 1.0f / sqrtf(x);
}

void mahony_update(float ax, float ay, float az,
                  float gx, float gy, float gz,
                  float mx, float my, float mz) {

    float recipNorm;
    float hx, hy, bx, bz;
    float halfvx, halfvy, halfvz, halfwx, halfwy, halfwz;
    float halfex, halfey, halfez;

    // Normalise accelerometer
    recipNorm = invSqrt(ax*ax + ay*ay + az*az);
    ax *= recipNorm; ay *= recipNorm; az *= recipNorm;

    // Normalise magnetometer
    recipNorm = invSqrt(mx*mx + my*my + mz*mz);
```

```

mx *= recipNorm; my *= recipNorm; mz *= recipNorm;

// Reference direction of Earth's magnetic field
hx = 2.0f*(mx*(0.5f - q2*q2 - q3*q3) + my*(q1*q2 - q0*q3) + mz*(q1*q3 +
q0*q2));
hy = 2.0f*(mx*(q1*q2 + q0*q3) + my*(0.5f - q1*q1 - q3*q3) + mz*(q2*q3 -
q0*q1));
bx = sqrtf(hx*hx + hy*hy);
bz = 2.0f*(mx*(q1*q3 - q0*q2) + my*(q2*q3 + q0*q1) + mz*(0.5f - q1*q1 -
q2*q2));

// Estimated direction of gravity and magnetic field
halfvx = q1*q3 - q0*q2;
halfvy = q0*q1 + q2*q3;
halfvz = q0*q0 - 0.5f + q3*q3;
halfwx = bx*(0.5f - q2*q2 - q3*q3) + bz*(q1*q3 - q0*q2);
halfwy = bx*(q1*q2 - q0*q3) + bz*(q0*q1 + q2*q3);
halfwz = bx*(q1*q3 + q0*q2) + bz*(0.5f - q1*q1 - q2*q2);

// Error is cross product of estimated and measured directions
halfex = (ay*halfvz - az*halfvy) + (my*halfwz - mz*halfwy);
halfey = (az*halfvx - ax*halfvz) + (mz*halfwx - mx*halfwz);
halfez = (ax*halfvy - ay*halfvx) + (mx*halfwy - my*halfwx);

// Apply integral feedback
integralFBx += TWO_KI * halfex * (1.0f / SAMPLE_FREQ);
integralFBy += TWO_KI * halfey * (1.0f / SAMPLE_FREQ);
integralFBz += TWO_KI * halfez * (1.0f / SAMPLE_FREQ);
gx += integralFBx;
gy += integralFBy;
gz += integralFBz;

// Apply proportional feedback
gx += TWO_KP * halfex;
gy += TWO_KP * halfey;
gz += TWO_KP * halfez;

// Integrate rate of change of quaternion
float dt = 1.0f / SAMPLE_FREQ;
q0 += (-q1*gx - q2*gy - q3*gz) * (0.5f * dt);
q1 += ( q0*gx + q2*gz - q3*gy) * (0.5f * dt);
q2 += ( q0*gy - q1*gz + q3*gx) * (0.5f * dt);
q3 += ( q0*gz + q1*gy - q2*gx) * (0.5f * dt);

```

```

// Normalise quaternion
recipNorm = invSqrt(q0*q0 + q1*q1 + q2*q2 + q3*q3);
q0 *= recipNorm; q1 *= recipNorm;
q2 *= recipNorm; q3 *= recipNorm;
}

Option 2: Fill the movement_t struct
Call this after mahony_update() each cycle:
c
#define RAD_TO_DEG 57.29577951f

movement_t pim448_to_movement(PIM448 &imu) {

    // Feed the filter (gyro must be in rad/s — convert if in deg/s)
    mahony_update(
        imu.getAccelX(), imu.getAccelY(), imu.getAccelZ(),
        imu.getGyroX(), imu.getGyroY(), imu.getGyroZ(), // ensure rad/s!
        imu.getMagX(), imu.getMagY(), imu.getMagZ()
    );

    movement_t m;

    // Quaternion components
    m.w = q0;
    m.i = q1;
    m.j = q2;
    m.k = q3;

    // Euler angles from quaternion
    m.roll = atan2f(2.0f*(q0*q1 + q2*q3),
        1.0f - 2.0f*(q1*q1 + q2*q2)) * RAD_TO_DEG;

    m.pitch = asinf(2.0f*(q0*q2 - q3*q1)) * RAD_TO_DEG;

    m.yaw = atan2f(2.0f*(q0*q3 + q1*q2),
        1.0f - 2.0f*(q2*q2 + q3*q3)) * RAD_TO_DEG;

    return m;
}

```

Key things to watch out for

- **Gyro units:** Mahony expects **rad/s**. If your PIM448 driver returns deg/s, multiply by `M_PI / 180.0f` before passing in.
- **Sample rate:** Set `SAMPLE_FREQ` to match how often you actually call the function — drift will accumulate if it's wrong.
- **Magnetometer calibration:** Hard-iron distortion will skew your yaw. You'll

want to capture min/max on all axes and subtract the offsets.

- **Gimbal lock:** `asinf` for pitch will have singularities near $\pm 90^\circ$. If you need full 3D rotation, work with the quaternion directly and skip Euler angles.

You said: Note that the PIM gets gyroscope data in degrees/s.

Note that the PIM gets gyroscope data in degrees/s. Does that change anything in the code?

24 Apr

Claude responded: Yes, just one change needed — convert the gyro inputs from deg/s to rad/s before passing them into `mahonyupdate()`.

Yes, just one change needed — convert the gyro inputs from deg/s to rad/s before passing them into `mahony_update()`. Update the call in `pim448_to_movement()`:

c

```
#define DEG_TO_RAD 0.01745329252f // M_PI / 180.0f

movement_t pim448_to_movement(PIM448 &imu) {

    mahony_update(
        imu.getAccelX(), imu.getAccelY(), imu.getAccelZ(),
        imu.getGyroX() * DEG_TO_RAD, // convert deg/s → rad/s
        imu.getGyroY() * DEG_TO_RAD,
        imu.getGyroZ() * DEG_TO_RAD,
        imu.getMagX(), imu.getMagY(), imu.getMagZ()
    );

    // ... rest unchanged
}
```

The `mahony_update()` function itself stays exactly as written — no changes there. Everything else (quaternion integration, Euler angle conversion) operates in radians already, so that one multiplication is the only fix needed.